PROJECT DOCUMENTATION

# MonuVista Website Project Report

System architecture, page descriptions, gesture technology, implementation, and deployment guide

**What this report provides**
**A practical reference for understanding the current website, preparing a project presentation, explaining design and technology choices, and publishing the source code through GitHub and Vercel.**
**Key conclusion: MonuVista is a browser-based heritage learning experience. It combines React pages and static monument information with Sketchfab-hosted 3D models and optional, local-in-the-browser hand tracking.**

| Project detail | Current implementation |
|---|---|
| Project name | MonuVista |
| Project type | Student heritage education and interactive exploration website |
| Application model | Client-side React single-page application (SPA) |
| Primary audience | Students and general visitors interested in Indian monuments |
| Prepared | 20 October 2026 |

Prepared from the current project source code and the features implemented in this workspace.

# How to use this report

Use the first half to understand what the website does and how it is built. Use the implementation and deployment sections when explaining hand tracking or publishing the site. The final sections provide a presentation outline, glossary, and official technical references.

# Contents

- 1. Project summary and goals
- 2. Intended users and educational positioning
- 3. Pages, navigation, and user journeys
- 4. Monument content and feature inventory
- 5. System architecture and data flow
- 6. Technology stack and why each choice fits
- 7. Hand gesture system in technical detail
- 8. Camera, privacy, permissions, and browser requirements
- 9. Voice guide, 3D viewer, and manual controls
- 10. Source code file map
- 11. Exact GitHub file checklist and publishing instructions
- 12. Quality checks, limits, and future work
- 13. Suggested presentation outline
- 14. Glossary and references

## Quick answer: what to upload to GitHub

Upload the application source folders and the root configuration files listed in Section 11. Keep all of public/mediapipe because it contains the hand model and WebAssembly runtime. Do not upload node_modules, dist, or .npm-cache. The .gitignore and vercel.json files have been added at the project root to make this easier.

Recommended host: Vercel, connected to the GitHub repository. The deployment uses npm run build, publishes dist, and uses vercel.json to send direct page requests back to the React app.

# 1. Project summary and goals

MonuVista is an interactive website for discovering Indian heritage monuments. A visitor can browse a curated list, read introductory history and architecture notes, open a monument detail page, listen to selected text, and launch a hosted 3D model. On the 3D page, users can choose mouse/touch interaction or enable a webcam-based gesture mode.

The project combines educational content with direct visual exploration. Its central design goal is to make monument information approachable and to let visitors inspect a model through familiar controls: move one open palm to rotate, move two open palms together or apart to zoom, and hold a fist to reset.

## Project objectives

- Discover: present a browsable catalogue of selected Indian monuments.
- Explain: provide concise context, historical periods, styles, facts, and architectural feature notes.
- Explore: embed 3D models so a visitor can rotate, zoom, and pan around a monument.
- Include: offer read-aloud controls and optional camera gestures alongside standard controls.
- Publish: build as a static frontend that can be deployed without a custom application server.

## Project scope

The current implementation is a frontend website with a JavaScript data catalogue. It does not contain user accounts, a database, a content management system, a quiz engine, a learning management system, or a custom 3D rendering backend. Those could be future additions, but they should not be described as existing features in a presentation.

# 2. Intended users and educational positioning

The About section explicitly describes MonuVista as a student heritage website. The best description is therefore a student-focused heritage learning and exploration website that is also usable by the general public. It is educational in purpose, but it is not currently a formal school platform or an LMS.

| Audience | How the current website helps |
|---|---|
| Students | Short overviews, dates, facts, architectural vocabulary, read-aloud support, and 3D inspection give students a visual starting point for research and presentations. |
| Teachers and presenters | The catalogue and feature notes can support a classroom discussion or a heritage presentation; the current source does not provide lesson plans or teacher accounts. |
| General visitors | Card-based browsing, familiar navigation, and optional 3D/voice controls support informal exploration. |
| Visitors using hands-free interaction | The gesture mode offers an alternative way to control the embedded model when the camera is available and permission is granted. |

## User-friendliness principles present in the source

- The home page links directly to monument cards and the Taj Mahal 3D experience.
- Each monument has a clear detail route and a visible link to its 3D route.
- Gesture mode is optional; mouse and touch controls remain available in the Sketchfab viewer.
- Camera start, pause, resume, disable, and reset controls are visible rather than hidden behind gestures.
- Status messages explain camera permission problems, missing cameras, tracking state, and detected gestures.
- Text-to-speech uses the browser/device speech service; no audio recordings are required.

# 3. Pages, navigation, and user journeys

| Route | Page component | Purpose |
|---|---|---|
| / | Home.jsx | Welcome, optional voice greeting, hero actions, featured monument cards, and feature summaries. |
| /monuments/:id | MonumentDetails.jsx | Monument image and overview, location and period, style, facts, feature details, read-aloud, and 3D link. |
| /monuments/:id/3d | Explorer3D.jsx | Hosted 3D viewer, interaction instructions, gesture controls, and architectural feature panel. |

Routes are declared in src/App.jsx and run inside BrowserRouter in src/main.jsx. Unknown monument identifiers redirect to the home page. A React single-page route changes the displayed component without requesting a different HTML file from the app itself; the hosting configuration must therefore serve index.html for direct requests to routes such as /monuments/taj-mahal/3d.

## Home page

- Navbar brand and links to Home, Monuments, Explore, Stories, and About Us anchors.
- Hero region with a welcome voice toggle, headline, descriptive copy, Explore Monuments link, and 3D Experience link.
- Decorative elephant illustration loaded from public/heritage-elephant-clean.png.
- Featured Monuments section generated from the monuments array in src/data/monuments.js.
- Feature strip describing history, 3D visual tours, curated insights, and image galleries.

## Monument detail page

- Loads a monument by the route id using getMonument(id).
- Displays remote monument image, place, historical period, overview, and architectural style.
- Shows the selected monument facts as cards and feature details as selectable topics.
- Read Aloud sends the overview or selected feature text to browser speech synthesis.
- Explore in 3D navigates to the matching model route.

### 3D Explorer page

- Loads the monument's Sketchfab model in an iframe through Sketchfab's JavaScript Viewer API.
- Shows short instructions, controls, live status, gesture legend, and architectural feature notes.
- Lets the visitor enable gesture control after the viewer is ready and after browser camera permission is granted.
- Retains normal Sketchfab mouse/touch interaction; hand tracking temporarily yields while a visitor manually interacts with the viewer.

### Typical visitor journey

1. Open Home and browse the five featured monuments.
2. Select a card to open its detail page and read facts or listen to the overview.
3. Select Explore in 3D.
4. Rotate/zoom/pan with the viewer's normal controls or choose Enable Hand Gesture and approve camera access.
5. Use one open palm to rotate, two open palms to zoom, or hold a fist to reset. Select Reset View at any time.

## 4. Monument content and feature inventory

The catalogue is a static JavaScript array in src/data/monuments.js. It is not loaded from a database or external content API. Each record includes an id, name, location, period, architecture style, remote image URL, short description, overview, Sketchfab embed URL, facts, and named architectural features with explanatory text.

| Monument | Location | Period in catalogue | 3D model |
|---|---|---|---|
| Taj Mahal | Agra, Uttar Pradesh | 1632-1653 | Sketchfab embedded model |
| Charminar | Hyderabad, Telangana | 1591 | Sketchfab embedded model |
| Qutub Minar | New Delhi | 1199-1220 | Sketchfab embedded model |
| Hampi | Karnataka | 14th-16th century | Sketchfab embedded model |
| Great Stupa in Sanchi | Sanchi, Madhya Pradesh | 3rd century BCE onwards | Sketchfab embedded model |

### Content model

The data shape lets one page template serve multiple monuments. The route id selects a record. Home maps every record into a MonumentCard; details and explorer pages use the same record, avoiding a separate hand-written page for each monument. FeaturePanel receives the selected record's features, so the text and labels adapt to the current monument.

### Media sources

The catalogue points monument images at Wikimedia Commons Special:FilePath URLs and 3D model embeds at Sketchfab. The homepage's decorative elephant files and the MediaPipe model/runtime are served locally from public/. External image/model availability and licensing remain dependent on the source services and their terms; verify attribution and use rights before public or commercial publication.

## 5. System architecture and data flow

MonuVista is a browser-first frontend. React renders pages and components; static JavaScript provides monument data; Vite bundles the application. The browser talks directly to external services for Sketchfab models and remote images. Camera frames are read by the browser and passed into the local MediaPipe task runtime. The project contains no application API server or database layer.

| Layer | Responsibility | Current implementation |
|---|---|---|
| Presentation | Render page layouts, cards, feature panels, control states, and responsive styling. | React components and src/styles.css |
| Navigation | Map URL paths to page components and provide in-app links. | React Router BrowserRouter, Routes, Route, Link, and useParams |
| Content | Store the five monument records and reusable gesture/camera calculations. | src/data/monuments.js and src/lib/gestureLogic.js |

| Layer | Responsibility | Current implementation |
|---|---|---|
| Camera input | Request permission, play video frames, and release camera tracks. | Browser MediaDevices API and HTMLVideoElement |
| Vision inference | Estimate up to two hand landmark sets from each processed video frame. | MediaPipe Tasks Vision, hand_landmarker.task, and local WASM runtime |
| 3D rendering | Display the model and expose camera position/target control. | Sketchfab hosted iframe and Viewer API |
| Speech | Speak welcome, overview, or feature text using available device voices. | Browser Web Speech API SpeechSynthesis |
| Build and hosting | Bundle source into deployable static files and serve routes. | Vite build, GitHub source repository, Vercel static hosting |

## Architecture map

| Visitor and browser | MonuVista application | Connected services |
|---|---|---|
| User opens route; uses links, mouse/touch, camera, and read-aloud. | React page components + Router + static monument data + gesture logic. | Wikimedia image URLs; Sketchfab Viewer API + hosted model; OS/browser speech voices. |
| Camera access is requested only after the user selects Enable Hand Gesture. | HTML video frames -> MediaPipe landmark inference -> gesture classifier -> camera calculations. | No MonuVista webcam upload endpoint exists in this codebase. |
| Browser sends camera look-at updates to the embedded model. | SketchfabViewer wraps the iframe; GestureController calls the viewer API. | Sketchfab renders and streams the 3D model. |

## Request and control flow

- Navigation flow: Browser URL -> BrowserRouter -> matching Route -> page component -> shared components and selected monument record.

- Detail flow: useParams reads :id -> getMonument finds a static record -> detail UI reads overview/facts/features -> links to the 3D route.

- 3D flow: model embed URL -> Sketchfab iframe/API script -> viewer initialization callback -> viewerready -> API reference passed into GestureController.

- Gesture flow: consented camera stream -> video element -> MediaPipe detects landmarks -> local rules classify motion -> camera position/target calculation -> Sketchfab setCameraLookAt.

- Speech flow: text from a component -> SpeechSynthesisUtterance -> browser/device voice engine -> spoken output.

# 6. Technology stack and why each choice fits

| Technology | Version / form | Why it is used here | Alternatives and trade-off |
|---|---|---|---|
| JavaScript (ES modules) | Application language | Works directly with browser APIs, React, MediaPipe, and Sketchfab's JavaScript API; the current project is small enough to keep a light setup. | TypeScript could add compile-time type checks as the data model and team grow; it is not currently configured. |
| React + React DOM | 19.1.0 | Reusable components separate pages, cards, feature controls, voice, and gestures; hooks manage page and device state. | A static HTML site would be simpler but less convenient for data-driven routed pages and stateful interactions. |
| React Router DOM | 7.8.0 | Provides BrowserRouter, dynamic :id paths, nested UI links, and client-side navigation for three page types. | Full-stack frameworks add server rendering/data features that this static catalogue does not currently require. |
| Vite | 7.1.0 | Provides the development server and production bundling; npm run build outputs dist/. | Webpack or other bundlers can do this too but would add configuration choices without a present need. |
| MediaPipe Tasks Vision | @mediapipe/tasks-vision ^0.10.21 | Provides a ready hand landmark task and WebAssembly/GPU/CPU runtime, avoiding training a custom hand model. | A custom model would require training data, model evaluation, and deployment work. A server vision API would add network latency and transmit camera frames. |
| Sketchfab Viewer API | External API version 1.12.1 | Hosts the detailed 3D assets and lets the site control the embedded viewer camera without maintaining a separate 3D engine or asset pipeline. | Three.js or another WebGL engine would allow full rendering control but require model files, optimization, and more implementation work. |
| Lucide React | ^0.542.0 | Consistent lightweight vector icons for navigation, actions, camera, volume, and gestures. | Text-only controls or a full icon font are possible; this package integrates with component markup. |
| CSS | src/styles.css | Custom site styling and responsive layout are implemented without a utility framework. | A CSS framework could speed a larger design system but would introduce conventions/dependencies not needed for the current site. |
| Browser MediaDevices API | getUserMedia | Prompts for a video stream after an explicit button action; no custom camera driver is needed. | Native/mobile SDKs would only be relevant for a separate installed mobile app. |
| Browser Web Speech API | speechSynthesis | Reads selected text without maintaining prerecorded audio files or a speech backend. | A hosted TTS service would offer voice consistency but would require network calls and service configuration. |

## Why there is no backend

Current monument content is bundled into the frontend, inference runs in the browser, the 3D models are hosted by Sketchfab, and speech synthesis is provided by the device/browser. Consequently, the application can be delivered as static assets. A backend would become useful if the product later needs accounts, teacher-managed lessons, editable content, progress tracking, user feedback storage, or protected API credentials.

# 7. Hand gesture system in technical detail

The gesture feature is an optional control layer for the 3D camera. It does not draw or render a separate 3D model. It estimates hand landmarks from webcam video and translates the landmark coordinates into camera position and target updates sent to the Sketchfab Viewer API.

## 7.1 Initialization and frame loop

- The user first selects Enable Hand Gesture; this user action starts camera initialization.

- The app checks navigator.mediaDevices.getUserMedia availability and requests video only, using the user-facing camera when available and ideal dimensions of 640 by 480. Audio is disabled.

- The HTML video element receives the MediaStream and begins playback.

- FilesetResolver loads MediaPipe's local vision WASM runtime from /mediapipe; HandLandmarker loads /mediapipe/hand_landmarker.task.

- The task is configured in VIDEO mode for up to two hands. Detection, presence, and tracking confidence thresholds are set to 0.65, 0.65, and 0.60.

- Initialization tries the GPU delegate first. If GPU setup throws, it retries with CPU.

- requestAnimationFrame schedules processing. detectForVideo runs only when the video has a new frame, using performance.now() as the frame timestamp.

- When the controller is disabled or unmounted, it cancels animation scheduling, closes the landmarker, stops every camera track, and clears the video source.

## 7.2 Landmarks and gesture classification

MediaPipe returns landmark coordinates for detected hands. The implementation uses the conventional 21-point hand layout: wrist index 0, middle-finger MCP index 9, index-finger tip 8/joint 6, middle tip 12/joint 10, ring tip 16/joint 14, pinky tip 20/joint 18, and palm width anchors 5 and 17.

| Recognized control | Recognition rule | Result |
|---|---|---|
| Open palm | At least three of four non-thumb finger tips are farther from the wrist than their paired joints, with a 1.08 multiplier; a thumb/index closure ratio below 0.4 rejects the hand as a pinch/unknown. | One open palm enables rotation. Two open palms enable zoom. |
| Fist | At most one of the four non-thumb fingers is extended and the thumb/index spacing has not triggered the pinch guard. | A stable fist held for 2.5 seconds resets the view. |
| Unknown | Landmarks are invalid, the pinch guard is met, or the finger pattern does not meet palm/fist rules. | No camera transform is applied; tracking waits for a recognized gesture. |

Gesture candidates must be recognized for three frames before becoming the active gesture. This short stability filter reduces accidental switches caused by a single noisy frame. If no hand is detected, tracking history is cleared so a newly appearing hand does not create a large jump.

## 7.3 One-palm rotation

The palm control point is the midpoint between the wrist (landmark 0) and middle-finger MCP (landmark 9). It is smoothed with alpha 0.45. The difference between consecutive smoothed positions becomes yaw and pitch input. A 0.0004 dead zone removes very small motion, then horizontal and vertical multipliers of 11 and 8 scale the input. Each camera frame is clamped to at most 0.14 radians of yaw and 0.10 radians of pitch change; overall pitch stays between -1.35 and +1.35 radians. These are current tuning values, not universal MediaPipe constants.

## 7.4 Two-palm zoom

When both detected hands classify as open palms, the app finds the center of each palm by averaging landmarks 0 and 9. It measures the distance between those centers and divides it by the average palm width from landmarks 5 and 17. Normalizing by hand size makes the zoom less dependent on how close the hands are to the webcam. Bring palms closer together to zoom in; spread them apart to zoom out.

- A stable two-palm gesture records a starting palm-gap ratio and the current camera distance.

- The current ratio is filtered with alpha 0.32 to reduce tracking jitter.

- A 0.015 dead zone is removed from ratio movement, then an exponential mapping with sensitivity 3 converts the change into a desired distance.

- The desired camera distance is clamped to a range from 0.04 times to 6 times the original camera distance.

- The camera moves 55 percent of the remaining distance toward the desired distance on each processed frame, and the Sketchfab API is updated with zero animation duration.

- The same zoom distance limits are passed to orbit logic so rotating after a zoom does not force the camera back to the original distance range.

```
desiredDistance = startDistance * exp((currentPalmGap - startPalmGap - deadZone) * sensitivity)
clamp(desiredDistance, originalDistance * 0.04, originalDistance * 6.0)
nextDistance = currentDistance + 0.55 * (desiredDistance - currentDistance)
```

The equation is a conceptual summary of the implementation; the actual dead-zone helper preserves the sign of the movement and clamps are applied in code. A 0.04 minimum is intentionally a very close zoom and may make a specific model appear cropped or enter the model, depending on the embedded model's camera setup.

### 7.5 Fist reset and manual control

- A fist must remain stable and be held for 2,500 ms before the reset runs.
- A reset cooldown of 1,800 ms prevents rapid repeated resets.
- The reset restores the camera position and target captured when the viewer first became ready and animates to them over 0.55 seconds.
- During iframe pointer interaction, hand tracking history is cleared and control yields to the Sketchfab viewer. After pointer release, the app reads the viewer camera again and waits a 1,100 ms grace period before resuming gesture reactions.
- Pause/Resume and Disable controls are available; reset is also available as a button regardless of gesture mode.

### Gesture interaction state sequence

| State | Entry/condition | Action |
|---|---|---|
| Off | Initial state, or user disables the control. | No camera stream or landmark frame loop is active. |
| Starting | User activates gesture mode. | Request permission, start video, load local runtime/model, initialize task. |
| Tracking / unknown | Task is ready but no stable palm/fist gesture is active. | Show status; wait for stable landmark pattern. |
| One palm | One open palm is stable for three frames. | Map palm center movement to camera yaw and pitch. |
| Two palms | Two open palms are stable for three frames. | Map normalized palm separation change to camera distance. |
| Fist hold | A stable fist is detected. | Start hold timer; reset once after 2.5 seconds. |
| Manual control | Pointer interaction begins inside viewer iframe. | Yield to Sketchfab controls, then sync camera on release. |
| Error | Permission, camera, or initialization fails. | Show a human-readable message; mouse/touch viewer controls remain usable. |

## 8. Camera, privacy, permissions, and browser requirements

The website does not secretly start the camera at page load. Gesture initialization is tied to the visible Enable Hand Gesture button. The request is for video only; the code sets audio to false. A browser permission prompt and camera indicator are controlled by the browser and operating system.

| Requirement | What the implementation needs |
|---|---|
| Secure origin | A production site must use HTTPS for camera access; localhost is suitable for local development. |
| Permission | The visitor must grant camera permission to the site. Denial is handled with a status message. |
| Camera hardware | A working built-in or external camera must be available. If none is found, the UI reports that condition. |
| Browser capabilities | The browser must provide MediaDevices/getUserMedia, video playback, requestAnimationFrame, WebAssembly, and the required WebGL/GPU path or CPU fallback. |
| Network | The page needs network access for hosted Sketchfab models/API and remote Wikimedia images. MediaPipe task and WASM assets are served by this site. |
| Device performance | Hand tracking processes new video frames in the browser; lower-end devices may deliver fewer processed frames or have higher latency. |

Within the current code, webcam frames are consumed by the browser's MediaPipe task and are not sent to a MonuVista backend because this repository has no such endpoint. This describes the application's code path, not a general privacy guarantee about the browser, operating system, or third-party services. Users can disable the feature; cleanup stops the stream tracks.

### Permission failure paths

- Browser API unavailable: camera access is reported as unavailable.
- NotAllowedError: permission was denied; mouse/touch controls remain available.
- NotFoundError: no camera was found; mouse/touch controls remain available.

- Other initialization error: status points the visitor to camera access and local tracking assets.
- Model/API initialization failure: the viewer error is displayed on the explorer page where possible.

# 9. Voice guide, 3D viewer, and manual controls

## Voice guide

VoiceGuide uses the browser's speechSynthesis service to speak a welcome sentence once for the current page load and exposes a Voice On/Voice Off control. MonumentDetails speaks the selected monument overview. FeaturePanel speaks the active feature name and its explanatory detail. The app cancels queued speech before starting a new overview or feature utterance. Available voice, language, and pronunciation depend on the device and browser; the project does not ship a recorded narration library.

## Sketchfab embedding

SketchfabViewer loads Sketchfab's version 1.12.1 JavaScript API dynamically. It extracts the model identifier from the embed URL, creates a Sketchfab client bound to an iframe, initializes that model, starts the viewer, and waits for viewerready before exposing the API to Explorer3D. GestureController reads the starting camera with getCameraLookAt and writes transforms with setCameraLookAt. Sketchfab hosts and renders the model; MonuVista controls camera values through the API.

## Normal mouse and touch interaction

The iframe remains interactive, with full-screen and autoplay permissions declared for the embedded viewer. Pointer down/up/cancel and blur events inform the parent page when the visitor is manually interacting. The website pauses gesture updates during direct model manipulation, then synchronizes the controller's stored camera after interaction ends. This prevents the webcam controller from fighting the user's mouse/touch input.

# 10. Source code file map

| Path | Purpose |
|---|---|
| index.html | Vite HTML entry point; supplies the root element and loads src/main.jsx. |
| package.json | Project metadata, npm scripts, and dependency declarations. |
| package-lock.json | Exact resolved npm dependency tree for repeatable installation. |
| vite.config.js | Vite configuration; enables automatic JSX transformation. |
| .gitignore | Excludes generated, local, and secret files from Git. |
| vercel.json | Rewrites client-side routes to index.html for direct URL visits and refreshes. |
| src/main.jsx | Mounts React and wraps the app in BrowserRouter; imports global CSS. |
| src/App.jsx | Defines site shell and the home, detail, and explorer routes. |
| src/styles.css | Global styles, typography, colors, layout, and responsive breakpoints. |
| src/data/monuments.js | Five static monument records, detail copy, remote image URLs, feature content, and Sketchfab model URLs. |
| src/lib/gestureLogic.js | Reusable landmark classification, palm-distance, camera orbit/zoom, smoothing, and dead-zone functions. |
| src/pages/Home.jsx | Homepage hero, voice toggle, cards, and feature summary strip. |
| src/pages/MonumentDetails.jsx | Dynamic monument detail page, facts, read-aloud, and feature panel. |
| src/pages/Explorer3D.jsx | Dynamic 3D page; connects Sketchfab viewer, gesture controls, and features. |
| src/components/Navbar.jsx | Brand mark and primary navigation links. |
| src/components/Footer.jsx | About text and project positioning. |
| src/components/MonumentCard.jsx | Reusable image, location, summary, and detail link for a monument. |
| src/components/FeaturePanel.jsx | Selectable architectural feature descriptions and read-aloud action. |
| src/components/VoiceGuide.jsx | Homepage welcome speech and voice toggle. |
| src/components/SketchfabViewer.jsx | Loads the Sketchfab API, initializes iframe viewer, and reports interaction/API state. |
| src/components/GestureController.jsx | Requests camera, initializes MediaPipe, classifies gestures, controls camera, and manages status/cleanup. |
| public/mediapipe/hand_landmarker.task | Local MediaPipe hand landmark model required at runtime. |
| public/mediapipe/vision_wasm_internal.js | JavaScript loader/runtime support for MediaPipe Vision tasks. |
| public/mediapipe/vision_wasm_internal.wasm | WebAssembly binary required by MediaPipe Vision tasks. |
| public/heritage-elephant*.png | Homepage decorative heritage illustration and preview/cutout variants stored with the project. |
| scripts/gesture-logic.test.js | Node built-in test suite for core gesture/camera logic. |
| docs/MonuVista_Project_Report_and_Deployment_Guide.docx | This report and GitHub/Vercel reference. |

## Package scripts

| Command | Purpose |
|---|---|
| npm run dev | Start local Vite development server. |
| npm run build | Create the production bundle in dist/. |
| npm run preview | Preview the local production build after building. |
| npm run test:gesture | Run the gesture-logic Node test file. |

# 11. Exact GitHub file checklist and publishing instructions

The source folder D:\MonuVista2 is the project root. The required deployment files are already organized in that folder. The newly added .gitignore prevents generated folders from being committed, and vercel.json handles React Router paths on Vercel.

## 11.1 Files and folders to include in GitHub

```
MonuVista2/
|-- .gitignore
|-- index.html
|-- package.json
|-- package-lock.json
|-- vite.config.js
|-- vercel.json
|-- src/
|   |-- App.jsx
|   |-- main.jsx
|   |-- styles.css
|   |-- components/  (all seven .jsx files)
|   |-- data/monuments.js
|   |-- lib/gestureLogic.js
|   `-- pages/  (Home, MonumentDetails, Explorer3D)
|-- public/  (all current image files)
|   `-- mediapipe/
|       |-- hand_landmarker.task
|       |-- vision_wasm_internal.js
|       `-- vision_wasm_internal.wasm
|-- scripts/gesture-logic.test.js
`-- docs/MonuVista_Project_Report_and_Deployment_Guide.docx
```

In simple terms: include all files in src, public, scripts, and docs, plus the six root files shown above. Do not select files inside node_modules, dist, or .npm-cache. Keep package-lock.json so the deployment platform installs the same dependency versions.

## 11.2 Files and folders not to upload

| Do not commit | Reason |
|---|---|
| node_modules/ | Downloaded packages; the host installs them from package.json and package-lock.json. |
| dist/ | Generated production output; Vercel creates it during the build. |
| .npm-cache/ | Local package cache; not application source. |
| .git/ | Git's private repository metadata; Git creates/manages it. |
| .env or secret files | Do not publish passwords, private keys, or tokens. There are no required environment variables in the current app. |

## 11.3 GitHub Desktop steps

1. Install GitHub Desktop and sign in to the GitHub account that should own the project.

2. Choose File > Add Local Repository and select D:\MonuVista2. If it says the folder is not a Git repository, use the create repository option for this existing folder. Do not move the source files into node_modules or dist.

3. Review the file list. Confirm src, public/mediapipe, scripts, docs, package.json, package-lock.json, index.html, vite.config.js, .gitignore, and vercel.json are included. Confirm node_modules, dist, and .npm-cache are excluded.

4. Enter a first commit message such as Initial MonuVista website source and choose Commit to main.

5. Choose Publish repository. Give the repository a clear name such as MonuVista. Choose public if the project needs to be openly visible for assessment; choose private if the instructor/team requires restricted access.

6. After publishing, open the repository on GitHub and confirm the file list includes public/mediapipe/hand_landmarker.task and vercel.json.

## 11.4 Deploy from GitHub to Vercel

1. Open Vercel and sign in with the same GitHub account.

2. Choose Add New > Project and import the MonuVista repository.

3. Set the project root directory to the repository root (the folder containing package.json).

4. Use the Vite defaults or verify: Framework Preset Vite, Install Command npm install (or npm ci), Build Command npm run build, Output Directory dist.

5. Choose Deploy. Wait for the Vercel build to finish and open the generated HTTPS address.

6. Test the home page, a detail page, a direct refresh on /monuments/taj-mahal/3d, Sketchfab load, Reset View, and camera permission on the HTTPS address.

7. To update later, commit and push changes to GitHub. Vercel rebuilds and publishes the connected branch automatically.

### Why Vercel for this project

Vercel is a suitable simple host for a Vite static frontend: it can build from a GitHub connection, publish the dist directory, and provide an HTTPS URL. HTTPS matters because browsers restrict getUserMedia camera access to secure contexts. vercel.json adds the SPA fallback required for direct navigation to React Router paths. Official Vercel guidance documents the Vite build flow and SPA rewrite configuration.

### Deployment checklist

| Check | Expected result |
|---|---|
| Root contains package.json | Vercel detects/builds the Vite app from the project root. |
| Build command | npm run build succeeds. |
| Output directory | dist. |
| Model/runtime assets | The production URLs /mediapipe/hand_landmarker.task and /mediapipe/vision_wasm_internal.wasm load successfully. |
| SPA rewrite | Refreshing /monuments/taj-mahal/3d returns the application rather than a host 404. |
| HTTPS camera | Browser permission prompt appears after Enable Hand Gesture is selected. |
| External embeds | Sketchfab viewer and model load; remote images load. |

## 12. Quality checks, limits, and future work

### Current quality checks in the project

- scripts/gesture-logic.test.js uses Node's built-in test runner to cover palm/fist classification, palm distance measurement, dead-zone/smoothing helpers, camera orbit bounds, and zoom bounds/direction.

- npm run build invokes Vite's production build and is the final static compilation check.

- Manual device checks remain important for permissions, camera lighting, two-hand visibility, GPU/CPU performance, Sketchfab availability, and browser-specific speech voices.

A code-level test cannot verify that a particular webcam, lighting condition, user hand size, browser GPU, or hosted Sketchfab model performs well. The project report therefore describes the implemented rules and a recommended manual checklist; it does not claim a live cross-device evaluation.

### Current constraints

- The catalogue is hard-coded; adding/editing content requires a source edit and redeployment.

- The app depends on third-party availability for Sketchfab models/API and Wikimedia images.

- The speech voice depends on the user's browser and operating system.

- Gesture classification uses geometric thresholds; hand orientation, occlusion, lighting, camera quality, and individual anatomy can affect reliability.

- Very close zoom can reveal the inside of a model or clip it; model camera bounds vary.

- A remote iframe can complicate browser-level camera permissions if hosting policy changes. The current iframe allow list is designed for autoplay/fullscreen/XR; webcam access is requested by the parent page, not by the iframe.

- Responsive layout breakpoints are present in CSS, but the actual experience should still be checked on phone/tablet sizes and with camera access on mobile browsers.

### Useful future work

- Add a project README with setup, controls, and attribution information.

- Add keyboard-accessible alternatives and visible focus states for all controls; test with screen readers and reduced-motion preferences.
- Add loading and retry states for slow model/runtime downloads and unavailable external embeds.
- Collect user testing feedback and tune gesture sensitivity separately for webcam sizes and screen distance.
- Consider a zoom focal point/target adjustment and model-specific safe camera bounds to improve close inspection.
- Move monument content into a content-management or data service only if editorial scale or frequent updates justify the added backend.
- Add quizzes, guided learning paths, progress tracking, teacher resources, or accounts only if the project is intended to become a formal learning platform.
- Add a privacy explanation near camera activation and a browser compatibility checklist for public release.

# 13. Suggested presentation outline

The following 12-slide sequence turns this report into a clear project presentation. It moves from the user problem to the implemented system, then explains the technical choices and a short demonstration.

| Slide | Suggested title | Main points / visual |
|---|---|---|
| 1 | MonuVista | Project name, student heritage learning purpose, one hero screenshot. |
| 2 | Problem and opportunity | Heritage information can be text-heavy; connect concise learning content to an interactive visual experience. |
| 3 | Goals and audience | Students and general visitors; discover, learn, explore, listen. |
| 4 | Website map | Home -> monument details -> 3D experience; show the three route patterns. |
| 5 | Pages and features | Home, detail, and explorer screenshots; identify cards, facts, audio, model, and controls. |
| 6 | System architecture | Browser, React/Vite frontend, static data/assets, MediaPipe pipeline, Sketchfab and Web APIs. |
| 7 | Technology stack | React, React Router, Vite, MediaPipe, Sketchfab API, CSS, Lucide, browser camera and speech APIs. |
| 8 | How hand tracking works | Camera permission -> video frame -> 21 landmarks -> rules -> camera update. |
| 9 | Gesture control design | One open palm rotates; two palms zoom; held fist resets; mouse/touch remains available. |
| 10 | Why these technologies | Browser-based inference, hosted 3D assets, reusable pages, static deployment; discuss trade-offs. |
| 11 | Testing, privacy, and limitations | Permission handling, local inference path, external services, device variability, next quality checks. |
| 12 | Demo and next steps | Show a detail page and 3D controls; proposed enhancements and conclusion. |

## Suggested live demonstration

- Open the Home page and choose a monument card.
- Read one overview/fact and demonstrate Read Aloud if speech is enabled.
- Open the 3D experience and first show normal mouse controls.
- Enable hand gestures, approve the browser prompt, and demonstrate rotation, two-palm zoom, and hold-fist reset.
- Finish by showing that Reset View is available as a normal button and gesture mode can be disabled.

# 14. Glossary

| Term | Plain-language explanation |
|---|---|
| SPA | Single-page application. The browser loads the app shell once and JavaScript switches the displayed page for client-side routes. |
| Route | A URL pattern that selects a page component, such as /monuments/:id. |
| Landmark | A named point on a detected hand, such as the wrist or a finger tip. |
| MediaPipe | A Google open-source machine-learning task/runtime used here to estimate hand landmarks from video frames. |
| WebAssembly (WASM) | A compact, browser-executable format used by the local MediaPipe vision runtime. |
| Inference | Running a trained model on an input frame to estimate results; this project uses inference, not model training. |
| Camera target | The point in 3D space that the Sketchfab camera looks toward. |
| Camera position | The 3D coordinates of the virtual camera. Changing position relative to target produces orbit or zoom. |
| Dead zone | A small threshold that ignores tiny coordinate changes so tracking noise does not move the camera. |
| Smoothing/filtering | Blending new landmark/zoom measurements with previous values to reduce jitter. |
| HTTPS / secure context | An encrypted browser origin required for camera permission on deployed sites. |
| Build output | Generated production files in dist/ that a static host serves; do not hand-edit or commit by default. |

# Official technical references

These references explain the browser and platform technologies used by the project. The implementation-specific behavior is documented from the current MonuVista source.

- React Quick Start
- React Router BrowserRouter
- React Router declarative routing
- Vite Getting Started
- Vite CLI and build command
- MediaPipe Hand Landmarker web sample
- MediaPipe Hand Landmarker task
- MDN MediaDevices getUserMedia
- MDN SpeechSynthesis
- Sketchfab Viewer API introduction
- Sketchfab Viewer API camera functions
- Vercel Vite deployment guide

## Source files consulted

The project-specific sections were prepared from index.html, package.json, package-lock.json, vite.config.js, src/App.jsx, src/main.jsx, src/styles.css, src/data/monuments.js, src/lib/gestureLogic.js, src/pages/*.jsx, src/components/*.jsx, scripts/gesture-logic.test.js, public/mediapipe/*, .gitignore, and vercel.json.